Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра интеллектуальных информационных технологий

Лабораторная работа №4

По дисциплине «Интеллектуальный анализ данных»

Тема: «Предобучение нейронных сетей с использованием RBM»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Шоева Е.Т.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: научиться осуществлять предобучение нейронных сетей с помощью RBM

Общее задание

1. Взять за основу нейронную сеть из лабораторной работы №3. Выполнить обучение с предобучением, используя стек ограниченных машин Больцмана (RBM – Restricted Boltzmann Machine), алгоритм которого изложен в лекции. Условие останова (например, по количеству эпох) при обучении отдельных слоев как RBM выбрать самостоятельно.

2. Сравнить результаты, полученные при
- обучении без предобучения (ЛР 3);
- обучении с предобучением, используя автоэнкодерный подход (ЛР3);
- обучении с предобучением, используя RBM.

3. Обучить модели на данных из ЛР 2, сравнить результаты по схеме из пункта 2;

4. Сделать выводы, оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Вариант:

№	Выборка	Тип задачи	Целевая переменная
11	https://archive.ics.uci.edu/dataset/27/credit+approval	классификация	+/-

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import f1_score, confusion_matrix
from sklearn.datasets import load_breast_cancer
from ucimlrepo import fetch_ucirepo



def load_credit_approval():
    dataset = fetch_ucirepo(id=27)
    X = dataset.data.features.copy()
    y = dataset.data.targets.copy()
    y = y.iloc[:, 0].map({'+': 1, '-': 0}).values
    for col in X.columns:
        if X[col].dtype == object:
```

```
                X[col] = X[col].replace('?', np.nan)
        cat_cols = X.select_dtypes(include=['object', 'str']).columns
        num_cols = X.select_dtypes(exclude=['object', 'str']).columns
        for col in num_cols:
            X[col] = pd.to_numeric(X[col], errors='coerce')
        X[num_cols] = SimpleImputer(strategy='mean').fit_transform(X[num_cols])
        X[cat_cols] = SimpleImputer(strategy='most_frequent').fit_transform(X[cat_cols])
        X = pd.get_dummies(X, columns=cat_cols)
        X = X.astype(float).values
        X = MinMaxScaler().fit_transform(X)
        return X, y


    def load_wdbc():
        data = load_breast_cancer()
        X = MinMaxScaler().fit_transform(data.data)
        return X, data.target


    class Classifier(nn.Module):
        def __init__(self, input_dim, hidden_dims, num_classes):
            super(Classifier, self).__init__()
            dims = [input_dim] + hidden_dims
            self.hidden_layers = nn.ModuleList(
                [nn.Linear(dims[i], dims[i + 1]) for i in range(len(hidden_dims))]
            )
            self.output_layer = nn.Linear(hidden_dims[-1], num_classes)
            self.activation = nn.Sigmoid()

        def forward(self, x):
            for layer in self.hidden_layers:
                x = self.activation(layer(x))
            return self.output_layer(x)


    class SingleAutoencoder(nn.Module):
```

```
    def __init__(self, input_dim, hidden_dim):
        super(SingleAutoencoder, self).__init__()
        self.encoder = nn.Linear(input_dim, hidden_dim)
        self.decoder = nn.Linear(hidden_dim, input_dim)
        self.activation = nn.Sigmoid()

    def forward(self, x):
        z = self.activation(self.encoder(x))
        return self.decoder(z), z



class RBM:
    def __init__(self, n_visible, n_hidden):
        self.W = torch.randn(n_visible, n_hidden) * 0.01
        self.vb = torch.zeros(n_visible)
        self.hb = torch.zeros(n_hidden)

    def hidden_probs(self, v):
        return torch.sigmoid(v @ self.W + self.hb)

    def visible_probs(self, h):
        return torch.sigmoid(h @ self.W.t() + self.vb)

    def train(self, data, epochs=100, lr=0.05, batch_size=32, k=1):
        n = data.shape[0]
        for epoch in range(epochs):
            perm = torch.randperm(n)
            for start in range(0, n, batch_size):
                v0 = data[perm[start:start + batch_size]]
                h0_prob = self.hidden_probs(v0)
                h = torch.bernoulli(h0_prob)
                for _ in range(k):
                    v_prob = self.visible_probs(h)
                    v = torch.bernoulli(v_prob)
                    h_prob = self.hidden_probs(v)
                    h = torch.bernoulli(h_prob)
                bs = v0.shape[0]
```

```
                    self.W += lr * (v0.t() @ h0_prob - v_prob.t() @ h_prob) /
bs
                    self.vb += lr * (v0 - v_prob).mean(dim=0)
                    self.hb += lr * (h0_prob - h_prob).mean(dim=0)

        def reconstruction_error(self, data):
            with torch.no_grad():
                rec = self.visible_probs(self.hidden_probs(data))
                return torch.mean((data - rec) ** 2).item()


    def train_classifier(model, X_train, y_train, X_test, y_test, epochs=1500,
lr=0.01):
        X_train_t = torch.tensor(X_train, dtype=torch.float32)
        y_train_t = torch.tensor(y_train, dtype=torch.long)
        X_test_t = torch.tensor(X_test, dtype=torch.float32)
        criterion = nn.CrossEntropyLoss()
        optimizer = optim.Adam(model.parameters(), lr=lr)
        loss_history = []
        for epoch in range(epochs):
            model.train()
            optimizer.zero_grad()
            loss = criterion(model(X_train_t), y_train_t)
            loss.backward()
            optimizer.step()
            loss_history.append(loss.item())
        model.eval()
        with torch.no_grad():
            preds = torch.argmax(model(X_test_t), dim=1).numpy()
        f1 = f1_score(y_test, preds, average='macro')
        cm = confusion_matrix(y_test, preds)
        return f1, cm, loss_history


    def pretrain_autoencoders(X_train, hidden_dims, epochs=300, lr=0.01):
        current_input = torch.tensor(X_train, dtype=torch.float32)
        dims = [X_train.shape[1]] + hidden_dims
        weights = []
```

```
        for i in range(len(hidden_dims)):
            ae = SingleAutoencoder(dims[i], dims[i + 1])
            criterion = nn.MSELoss()
            optimizer = optim.Adam(ae.parameters(), lr=lr)
            for epoch in range(epochs):
                optimizer.zero_grad()
                out, _ = ae(current_input)
                loss = criterion(out, current_input)
                loss.backward()
                optimizer.step()
            ae.eval()
            with torch.no_grad():
                _, z = ae(current_input)
            weights.append((ae.encoder.weight.data.clone(),
ae.encoder.bias.data.clone()))
            current_input = z
        return weights



    def pretrain_rbms(X_train, hidden_dims, epochs=100, lr=0.05):
        current_input = torch.tensor(X_train, dtype=torch.float32)
        dims = [X_train.shape[1]] + hidden_dims
        weights = []
        for i in range(len(hidden_dims)):
            rbm = RBM(dims[i], dims[i + 1])
            rbm.train(current_input, epochs=epochs, lr=lr)
            weights.append((rbm.W.t().contiguous().clone(), rbm.hb.clone()))
            with torch.no_grad():
                current_input = rbm.hidden_probs(current_input)
        return weights



    def  build_pretrained_classifier(input_dim,  hidden_dims,  num_classes,
weights):
        model = Classifier(input_dim, hidden_dims, num_classes)
        for i, (w, b) in enumerate(weights):
            model.hidden_layers[i].weight.data = w.clone()
            model.hidden_layers[i].bias.data = b.clone()
```

```
        return model


    def plot_loss_curves(losses, dataset_name):
        plt.figure(figsize=(8, 6))
        for name, history in losses.items():
            plt.plot(history, label=name)
        plt.xlabel('Эпоха')
        plt.ylabel('Loss')
        plt.title(f'Сходимость обучения: {dataset_name}')
        plt.legend()
        plt.tight_layout()
        plt.savefig(f'lab4_loss_{dataset_name}.png')
        plt.show()


    def plot_confusion_matrices(cms, dataset_name):
        fig, axes = plt.subplots(1, len(cms), figsize=(5 * len(cms), 4.5))
        for ax, (name, cm) in zip(axes, cms.items()):
            sns.heatmap(cm,   annot=True,   fmt='d',   cmap='Blues',   ax=ax,
cbar=False)
            ax.set_title(name)
            ax.set_xlabel('Предсказано')
            ax.set_ylabel('Истина')
        fig.suptitle(dataset_name)
        plt.tight_layout()
        plt.savefig(f'lab4_cm_{dataset_name}.png')
        plt.show()


    def plot_f1_comparison(results_mean, results_std):
        datasets = list(results_mean.keys())
        methods = list(next(iter(results_mean.values())).keys())
        x = np.arange(len(datasets))
        width = 0.25
        plt.figure(figsize=(9, 6))
        for i, m in enumerate(methods):
            values = [results_mean[d][m] for d in datasets]
```

```
            errors = [results_std[d][m] for d in datasets]
            plt.bar(x + (i - 1) * width, values, width, yerr=errors, capsize=4,
label=m)
        plt.xticks(x, datasets)
        plt.ylabel('F1-score (macro)')
        plt.title('Сравнение методов обучения (среднее по запускам)')
        plt.legend()
        plt.tight_layout()
        plt.savefig('lab4_f1_comparison.png')
        plt.show()



    def run_single(X_train, X_test, y_train, y_test, input_dim, hidden_dims,
num_classes, seed):
        f1s, cms, losses = {}, {}, {}


        torch.manual_seed(seed)
        model = Classifier(input_dim, hidden_dims, num_classes)
        f1s['Без предобучения'], cms['Без предобучения'], losses['Без
предобучения'] = train_classifier(
            model, X_train, y_train, X_test, y_test
        )


        torch.manual_seed(seed)
        ae_weights = pretrain_autoencoders(X_train, hidden_dims)
        model = build_pretrained_classifier(input_dim, hidden_dims,
num_classes, ae_weights)
        f1s['Автоэнкодер'], cms['Автоэнкодер'], losses['Автоэнкодер'] =
train_classifier(
            model, X_train, y_train, X_test, y_test
        )


        torch.manual_seed(seed)
        rbm_weights = pretrain_rbms(X_train, hidden_dims)
        model = build_pretrained_classifier(input_dim, hidden_dims,
num_classes, rbm_weights)
        f1s['RBM'], cms['RBM'], losses['RBM'] = train_classifier(
            model, X_train, y_train, X_test, y_test
        )
        return f1s, cms, losses
```

```
    def run_experiment(dataset_name, X, y, hidden_dims, seeds=(0, 1, 2, 3,
4)):
        num_classes = len(np.unique(y))
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42, stratify=y
        )
        print(f"\n=== {dataset_name} ===")

        all_f1 = {}
        first_cms, first_losses = None, None
        for idx, seed in enumerate(seeds):
            f1s, cms, losses = run_single(
                X_train, X_test, y_train, y_test, X.shape[1], hidden_dims,
num_classes, seed
            )
            if idx == 0:
                first_cms, first_losses = cms, losses
                for name in f1s:
                    print(f"[seed {seed}] {name}: F1 = {f1s[name]:.4f}")
                    print("Confusion matrix:\n", cms[name])
            for name, value in f1s.items():
                all_f1.setdefault(name, []).append(value)

        mean = {k: float(np.mean(v)) for k, v in all_f1.items()}
        std = {k: float(np.std(v)) for k, v in all_f1.items()}

        print(f"\nСреднее F1 по {len(seeds)} запускам:")
        for name in mean:
            print(f"{name}: {mean[name]:.4f} ± {std[name]:.4f}")
        print(f"Разница F1 (Автоэнкодер - Без предобучения):
{mean['Автоэнкодер'] - mean['Без предобучения']:.4f}")
        print(f"Разница F1 (RBM - Без предобучения): {mean['RBM'] - mean['Без
предобучения']:.4f}")
        print(f"Разница F1 (RBM - Автоэнкодер): {mean['RBM'] -
mean['Автоэнкодер']:.4f}")

        plot_loss_curves(first_losses, dataset_name)
```

```
        plot_confusion_matrices(first_cms, dataset_name)
        return mean, std


    hidden_dims = [32, 16, 8]
    results_mean = {}
    results_std = {}

    X_credit, y_credit = load_credit_approval()
    results_mean['Credit Approval'], results_std['Credit Approval'] = run_experiment(
        "Credit Approval", X_credit, y_credit, hidden_dims
    )

    X_wdbc, y_wdbc = load_wdbc()
    results_mean['WDBC'], results_std['WDBC'] = run_experiment("WDBC", X_wdbc, y_wdbc, hidden_dims)

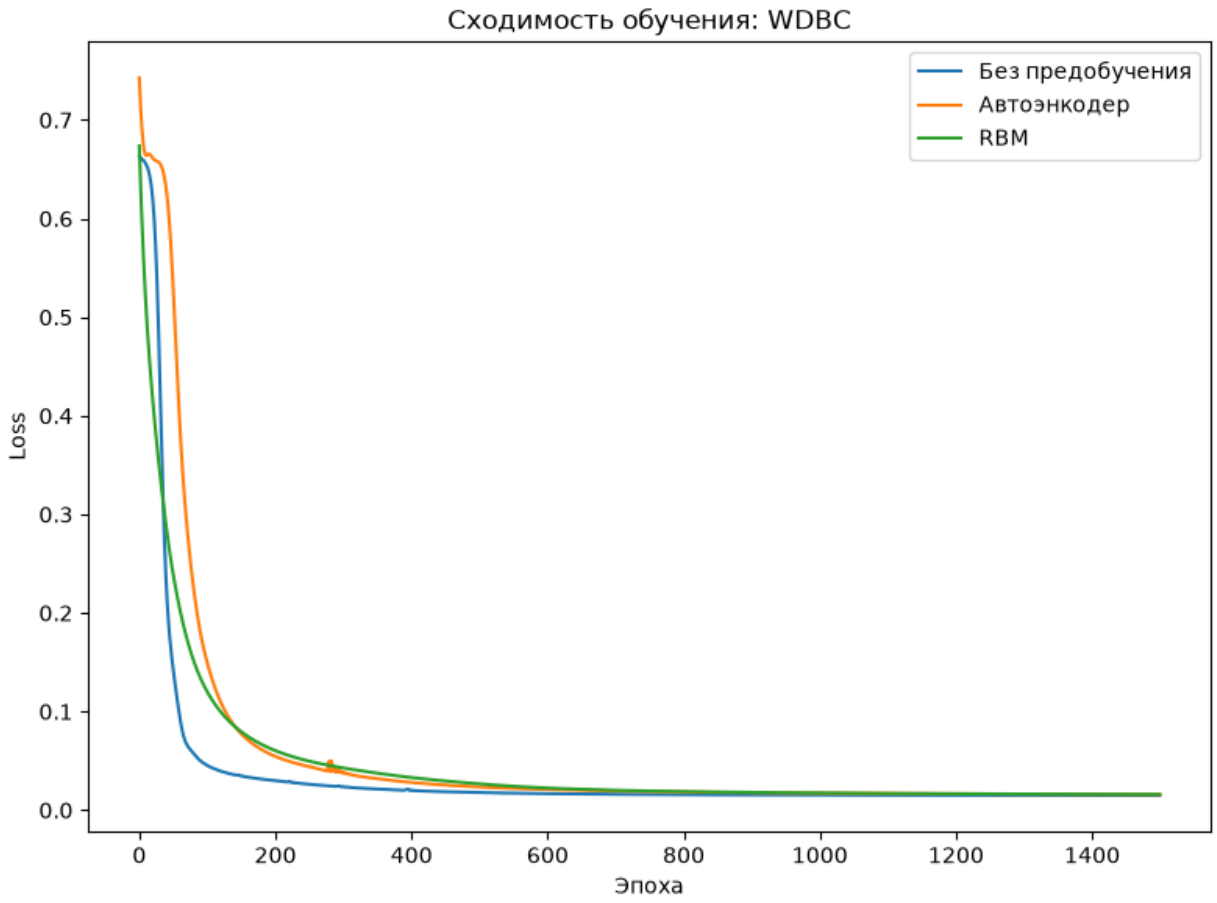
    plot_f1_comparison(results_mean, results_std)
```

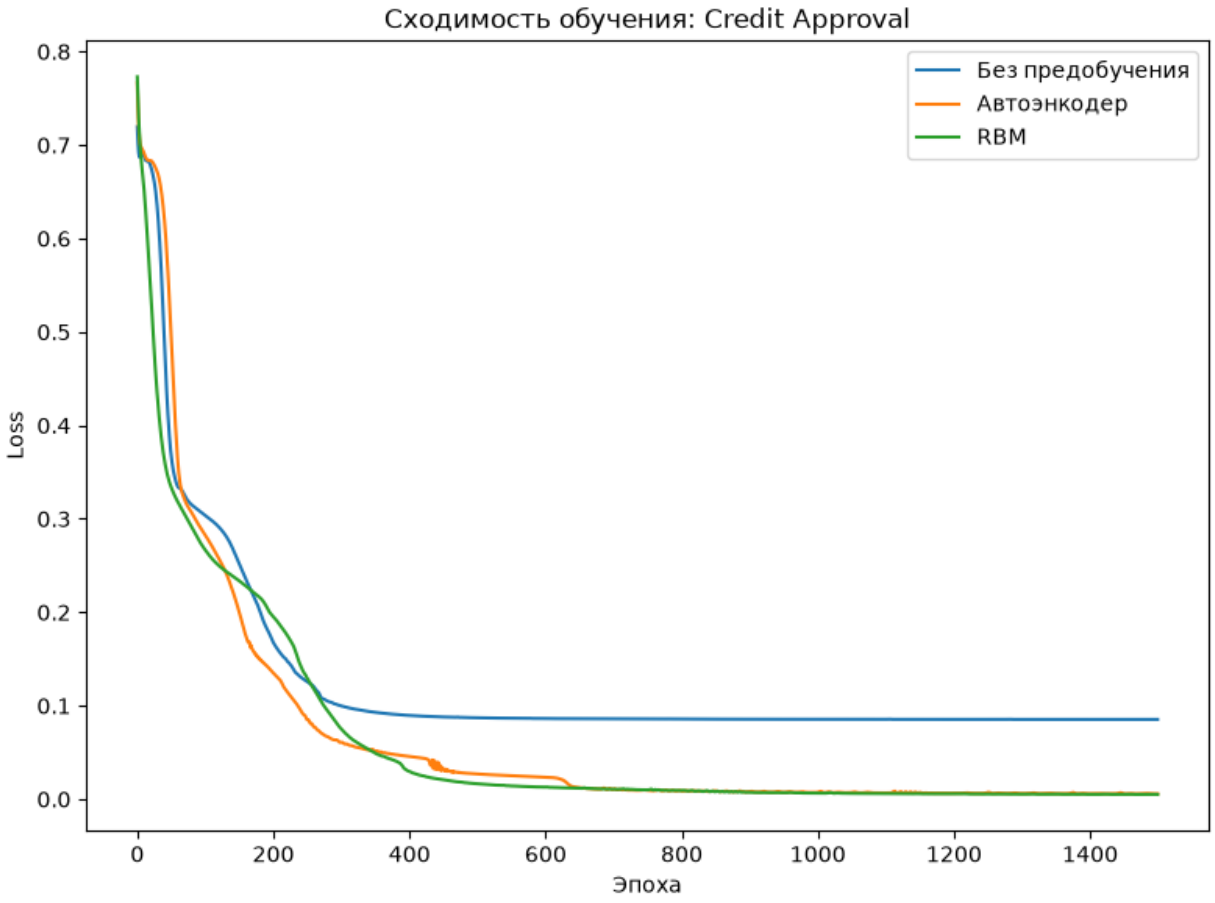
```
=== Credit Approval ===
[seed 0] Без предобучения: F1 = 0.8435
Confusion matrix:
 [[70  7]
 [14 47]]
[seed 0] Автоэнкодер: F1 = 0.8520
Confusion matrix:
 [[69  8]
 [12 49]]
[seed 0] RBM: F1 = 0.8083
Confusion matrix:
 [[65 12]
 [14 47]]

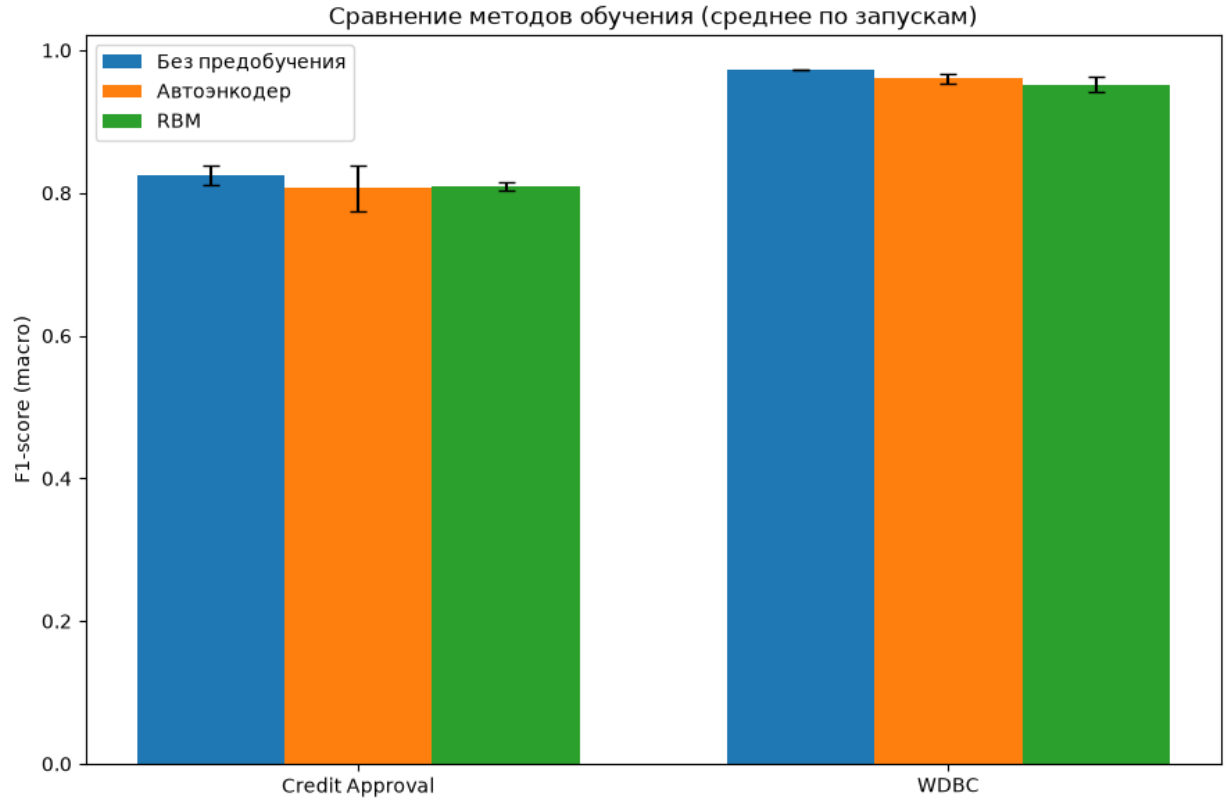
Среднее F1 по 5 запускам:
Без предобучения: 0.8246 ± 0.0136
Автоэнкодер: 0.8069 ± 0.0323
RBM: 0.8092 ± 0.0059
Разница F1 (Автоэнкодер - Без предобучения): -0.0177
Разница F1 (RBM - Без предобучения): -0.0155
Разница F1 (RBM - Автоэнкодер): 0.0022
```

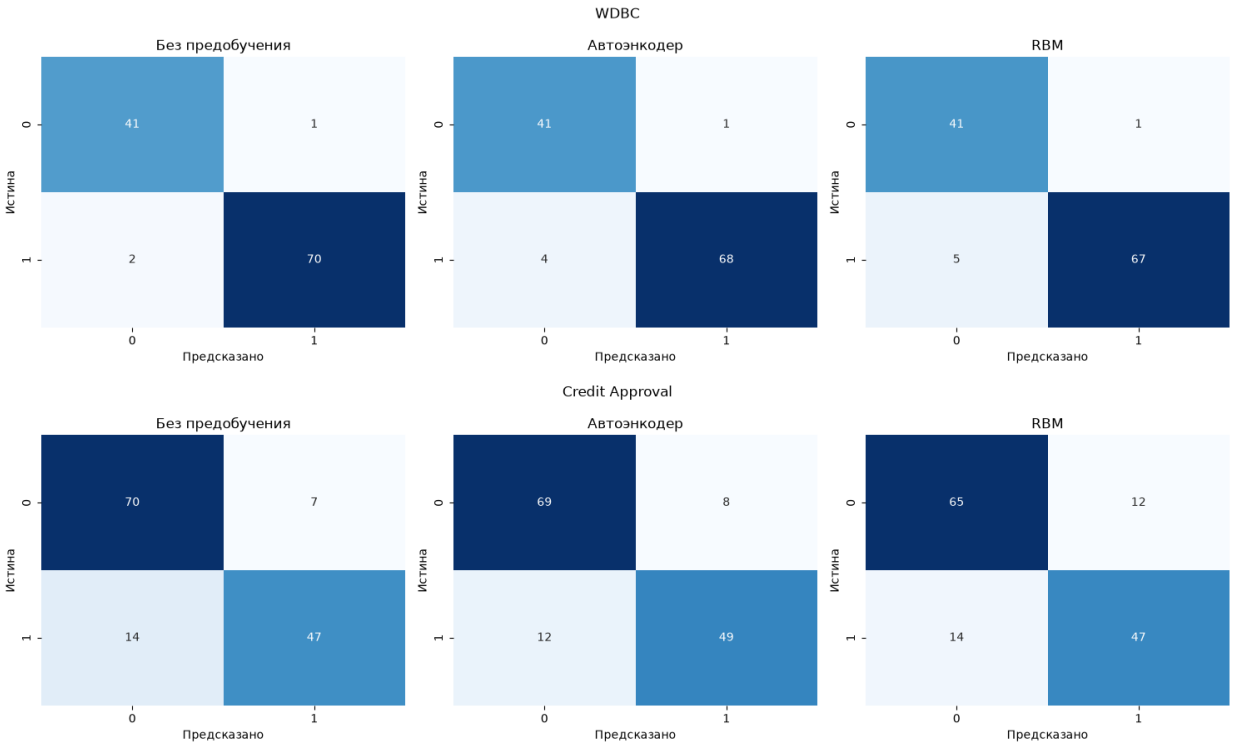
```
=== WDBC ===
[seed 0] Без предобучения: F1 = 0.9719
Confusion matrix:
 [[41  1]
 [ 2 70]]
[seed 0] Автоэнкодер: F1 = 0.9535
Confusion matrix:
 [[41  1]
 [ 4 68]]
[seed 0] RBM: F1 = 0.9445
Confusion matrix:
 [[41  1]
 [ 5 67]]

Среднее F1 по 5 запускам:
Без предобучения: 0.9719 ± 0.0000
Автоэнкодер: 0.9609 ± 0.0069
RBM: 0.9518 ± 0.0106
Разница F1 (Автоэнкодер - Без предобучения): -0.0110
Разница F1 (RBM - Без предобучения): -0.0201
Разница F1 (RBM - Автоэнкодер): -0.0091
```


Сходимость обучения: WDBC
Без предобучения
Автоэнкодер
RBM
Loss
Эпоха
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0
200
400
600
800
1000
1200
1400


Сходимость обучения: Credit Approval
Без предобучения
Автоэнкодер
RBM
Loss
Эпоха
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0
200
400
600
800
1000
1200
1400





Предобучение (автоэнкодером и стеком RBM) не улучшило качество классификации относительно обучения с нуля: на WDBC F1 составил 0.972 (без предобучения), 0.961 (автоэнкодер), 0.952 (RBM); на Credit Approval 0.825, 0.807 и 0.809 соответственно. Различия лежат в пределах разброса между запусками. Предобучение ускоряет начальную сходимость, но при малых выборках (569 и 690 объектов) и неглубокой сети (3 скрытых слоя) преимуществ по качеству не даёт. Оно эффективно при глубоких сетях и большом объёме данных.

Вывод: научилась осуществлять предобучение нейронных сетей с помощью RBM